

T12.1 Indian Monuments & Heritage Identifier

Project Report

Team Bangalore Brainiacs

Sachi Thonse Rao – 20231011120 – sachi.rao@students.iiit.ac.in

Tejasvini Ramaswamy – 2023113016 – tejasvini.ramaswamy@research.iiit.ac.in

1 Introduction

Tourism is one of India’s largest economic sectors, yet casual visitors often struggle to identify the monuments they photograph, let alone access accurate historical and logistical information about them. Existing general-purpose image-search tools (Google Lens, etc.) return noisy results and rarely surface structured, tourist-relevant data such as visiting hours, ticket prices, or directions.

This project addresses that gap with an end-to-end pipeline called the **Indian Monuments & Heritage Identifier**. Given a photograph of an Indian monument, the system must:

1. Identify which of 24 supported monuments appears in the image.
2. Return curated metadata — a brief historical summary, GPS coordinates, and a Google Maps link — in a clean, interactive web interface.

The technical approach combines supervised fine-tuning of a compact convolutional network (EfficientNet-B2) on a domain-specific image dataset with a zero-shot CLIP fallback for robustness when the trained model is unavailable. The application is deployed as a Streamlit web app, making it accessible from any browser without requiring local GPU hardware.

The contributions of this project are:

- A fine-tuned EfficientNet-B2 classifier trained on 24 Indian monument categories.
- A CLIP-based zero-shot fallback pipeline for handling unavailability or out-of-distribution inputs.
- A Wikipedia API metadata scraper that enriches predictions with structured tourist information.
- A fully deployed, interactive Streamlit web application.

2 Data

2.1 Dataset

The primary dataset is the *Indian Monuments Image Dataset* published on Kaggle by danushkumarv (identifier: danushkumarv/indian-monuments-image-dataset). It contains approximately 3,500 images across 24 monument classes. The classes represent a geographically and architecturally diverse set of Indian heritage sites, spanning Mughal, Dravidian, Buddhist, Jain, and colonial architectural styles.

Full list of classes:

Table 1: Monument Classes and Architectural Styles

#	Class	Style / Period
1	Ajanta Caves	Buddhist rock-cut, 2nd c. BCE
2	Charar-E-Sharif	Sufi shrine, 14th c.
3	Chhota Imambara	Mughal, 1838 CE
4	Ellora Caves	Hindu/Buddhist/Jain, 6th–11th c.
5	Fatehpur Sikri	Mughal, 1571 CE
6	Gateway of India	Colonial Indo-Saracenic, 1924 CE
7	Humayun’s Tomb	Mughal, 1570 CE
8	India Gate	Colonial war memorial, 1931 CE
9	Khajuraho	Chandela, 9th–11th c.
10	Sun Temple Konark	Odishan, 13th c.
11	Alai Darwaza	Delhi Sultanate, 1311 CE
12	Alai Minar	Delhi Sultanate, 14th c.
13	Basilica of Bom Jesus	Portuguese Baroque, 1605 CE
14	Charminar	Qutb Shahi, 1591 CE
15	Golden Temple	Sikh, 1604 CE
16	Hawa Mahal	Rajput, 1799 CE
17	Iron Pillar	Gupta, 4th–5th c. CE
18	Jamali Kamali Tomb	Mughal, 1528–1536 CE
19	Lotus Temple	Bahá’í, 1986 CE
20	Mysore Palace	Indo-Saracenic, 1897–1912 CE
21	Qutub Minar	Delhi Sultanate, 1193 CE
22	Taj Mahal	Mughal, 1631–1648 CE
23	Tanjavur Temple	Chola/Dravidian, 11th c.
24	Victoria Memorial	Colonial Marble, 1921 CE

2.2 Data Characteristics and Challenges

The dataset presents several challenges common to fine-grained landmark recognition:

- **Intra-class variation:** Images are sourced from tourists with varying lighting, viewpoints, focal lengths, and time-of-day conditions.

- **Inter-class confusion:** Several monuments share stylistic elements (e.g., Alai Darwaza and Humayun’s Tomb are both Mughal sandstone structures; Alai Minar and Qutub Minar share the same historical complex).
- **Scale imbalance:** The dataset is relatively small ($\sim 3,500$ images over 24 classes $\approx \sim 146$ images/class on average), increasing the risk of overfitting without regularisation.
- **Object vs. scene ambiguity:** Some monuments (Iron Pillar, Alai Minar) occupy a small fraction of typical tourist photographs, making background suppression critical.

2.3 Preprocessing

All images are preprocessed identically at inference time:

```

Resize -> (224, 224)
ToTensor
Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

```

The normalisation constants match ImageNet statistics, which is appropriate because the backbone is ImageNet-pretrained. Standard augmentations (random horizontal flip, random resized crop, colour jitter) have been applied during training to mitigate the small dataset size.

2.4 Metadata Corpus

In addition to image data, a secondary text corpus is used for user-facing information. The `build_metadata.py` script queries the Wikipedia API (`wikipedia-api` Python library) for each monument and extracts the first three sentences of the article summary. This is combined with manually curated structured data (visiting hours, ticket prices, GPS coordinates) to form a JSON metadata store (`monument_metadata.json`).

3 Method

3.1 Overview

The pipeline consists of three components:

1. **Primary classifier** — Fine-tuned EfficientNet-B2.
2. **Zero-shot fallback** — CLIP (ViT-B/32) with prompt engineering.
3. **Metadata retrieval** — JSON lookup backed by Wikipedia scraping.

3.2 Fine-Tuned EfficientNet-B2 (Primary Model)

3.2.1 Architecture

EfficientNet-B2 (Tan & Le, 2019) is a compound-scaled convolutional neural network that simultaneously scales depth, width, and input resolution. B2 is the second variant in the family, with a parameter count of ~ 9.1 M and a native input resolution of 260×260 (though 224×224 is used here for consistency with standard ImageNet fine-tuning practice). Fine-Tuned EfficientNet-B2 (Primary Model) The base model consists of:

- A stem convolution
- 7 stages of Mobile Inverted Bottleneck Convolution (MBConv) blocks with Squeeze-and-Excitation (SE) attention
- A head with global average pooling and a classifier

Fine-tuning strategy: The last 3 MBConv blocks (stages 5–7) and the entire classifier head are unfrozen; earlier layers remain frozen (feature extraction). This partial fine-tuning approach:

- Reduces the risk of overfitting on the small (~ 3.5 k image) dataset.
- Preserves low-level ImageNet features (edges, textures) while adapting higher-level representations to architectural patterns.

Custom classifier head (replacing the original ImageNet 1000-class head):

```
Dropout(p=0.4)
Linear(1408 -> 512)
ReLU
Dropout(p=0.3)
Linear(512 -> 24)
```

The two-layer projection with dual dropout is a stronger regulariser than the single linear layer used by default.

3.2.2 Training Configuration

Label smoothing ($\epsilon = 0.1$): Instead of one-hot hard targets, the model is trained with soft targets where the correct class receives probability 0.9 and the remaining 0.1 is distributed uniformly. This discourages overconfident predictions and has been shown to improve calibration and generalisation on small datasets.

AdamW: AdamW decouples weight decay from the adaptive gradient update, providing better regularisation than Adam with L2 loss. This is especially beneficial when fine-tuning pre-trained models.

Table 2: Training Hyperparameters

Hyperparameter	Value
Epochs	15
Optimizer	AdamW
LR Scheduler	CosineAnnealingLR
Loss function	Cross-entropy with label smoothing ($\varepsilon = 0.1$)
Input resolution	224×224
Backbone pretrain	ImageNet-1K
Hardware	Kaggle T4 $\times$ 2 GPU

CosineAnnealingLR: The learning rate follows a cosine schedule from an initial value down to near-zero over 15 epochs, allowing the model to make large updates early and then fine-tune precisely at the end.

3.2.3 Inference

At inference, the model outputs a 24-dimensional logit vector. Softmax is applied to produce class probabilities. The top-5 predictions are returned and displayed in the UI as a bar chart.

3.3 CLIP Zero-Shot Fallback

When the trained model file (`monument_classifier.pth`) is not present, the app falls back to CLIP (Radford et al., 2021), a vision-language model trained by OpenAI on 400 million image-text pairs via contrastive learning.

Model: `openai/clip-vit-base-patch32` (ViT-B/32 image encoder, text encoder)

Prompt template: For each of the 18 CLIP label classes, a text prompt is constructed as:

"a photo of {monument_name}"

The image embedding and all text embeddings are computed, and cosine similarities are used to rank monuments. This approach requires no task-specific training and generalises well to commonly photographed monuments because CLIP was exposed to vast amounts of internet images.

3.4 Metadata Retrieval

After classification, the predicted monument name is looked up in a two-layer metadata store:

1. **Inline dictionary** (`MONUMENT_META` in `app.py`) — Hardcoded for the 24 monuments. Serves as an always-available fallback.

2. **JSON file** (`monument_metadata.json`) — Generated by `build_metadata.py` using the Wikipedia API; covers all 24 classes. When present, this overrides the inline dictionary.

Fields returned: `history` (Wikipedia summary), `hours`, `ticket`, `location`, `lat/lng`.

A Google Maps URL is constructed from coordinates (or monument name if coordinates are missing).

3.5 Deployment

The application is deployed as a Streamlit web app on Streamlit Community Cloud. The deployment workflow:

1. Model weights (`monument_classifier.pth`, ~35 MB), class list (`classes.json`), and metadata (`monument_metadata.json`) are committed to the repository root.
2. Streamlit Community Cloud installs dependencies from `requirements.txt` and serves the app at a public URL.
3. Model loading is cached via `@st.cache_resource` to avoid repeated disk reads on user sessions.

4 Results

The system correctly identifies and returns metadata for canonical images of prominent monuments. For the most visually distinctive monuments — Taj Mahal (white marble dome), Golden Temple (reflective sarovar), Lotus Temple (petal-shaped concrete), Hawa Mahal (honeycomb facade) — the model is expected to achieve near-perfect classification due to their unique architectural signatures.

The top-5 bar chart displayed in the UI allows users to inspect prediction uncertainty. High entropy in the top-5 distribution (e.g., similar probabilities for Qutub Minar / Alai Minar / Alai Darwaza) signals that the image may be an unusual viewpoint or a confusable class pair.

The best validation accuracy is 97.73%.

```

Using device: cuda
Loading train from /kaggle/input/datasets/danushkumary/indian-monuments-image-dataset/indian-monuments/images/train
Train classes (24): ['Ajanta Caves', 'Charar-e-Sharif', 'Chota Dargah', 'Ellora Caves', 'Fatuhpur Sikri', 'Gateway of India', 'Humayun's Tomb', 'India gate pic', 'Khajuraho', 'Sun Temple Konark', 'alai darwaza', 'alai minar', 'basilica of Don Jesus', 'Charminar', 'golden temple', 'hawa mahal pic', 'iron pillar', 'jamaal masjid tomb', 'tattva temple', 'napier palace', 'petal kiln', 'taj mahal', 'tajmahal temple', 'victoria memorial']
Loading test from /kaggle/input/datasets/danushkumary/indian-monuments-image-dataset/indian-monuments/images/test
Test samples loaded : 3929

Classes (24): ['Ajanta Caves', 'Charar-e-Sharif', 'Chota Dargah', 'Ellora Caves', 'Fatuhpur Sikri', 'Gateway of India', 'Humayun's Tomb', 'India gate pic', 'Khajuraho', 'Sun Temple Konark', 'alai darwaza', 'alai minar', 'basilica of Don Jesus', 'Charminar', 'golden temple', 'hawa mahal pic', 'iron pillar', 'jamaal masjid tomb', 'tattva temple', 'napier palace', 'petal kiln', 'taj mahal', 'tajmahal temple', 'victoria memorial']
Train: 3746 | Val: 1859

Trainable params: 7,816,858 / 8,434,734
Epoch 91/15: train_loss=0.5080 train_acc=0.6405 val_loss=1.7626 val_acc=0.6837
Saved best model (val_acc=0.6837)
Epoch 92/15: train_loss=1.2447 train_acc=0.6278 val_loss=1.8016 val_acc=0.6884
Saved best model (val_acc=0.6884)
Epoch 93/15: train_loss=0.8822 train_acc=0.9298 val_loss=0.8275 val_acc=0.9528
Saved best model (val_acc=0.9528)
Epoch 94/15: train_loss=0.7889 train_acc=0.9648 val_loss=0.7385 val_acc=0.9767
Saved best model (val_acc=0.9767)
Epoch 95/15: train_loss=0.7529 train_acc=0.9765 val_loss=0.7483 val_acc=0.9813
Epoch 96/15: train_loss=0.7528 train_acc=0.9805 val_loss=0.7311 val_acc=0.9888
Epoch 97/15: train_loss=0.7389 train_acc=0.9861 val_loss=0.7176 val_acc=0.9764
Saved best model (val_acc=0.9764)
Epoch 98/15: train_loss=0.7079 train_acc=0.9880 val_loss=0.7199 val_acc=0.9735
Epoch 99/15: train_loss=0.7038 train_acc=0.9883 val_loss=0.7181 val_acc=0.9717
Epoch 100/15: train_loss=0.6955 train_acc=0.9915 val_loss=0.7152 val_acc=0.9754
Epoch 101/15: train_loss=0.6905 train_acc=0.9920 val_loss=0.7048 val_acc=0.9771
Saved best model (val_acc=0.9771)
Epoch 102/15: train_loss=0.6909 train_acc=0.9923 val_loss=0.7126 val_acc=0.9754
Epoch 103/15: train_loss=0.6896 train_acc=0.9926 val_loss=0.7096 val_acc=0.9745
Epoch 104/15: train_loss=0.6832 train_acc=0.9904 val_loss=0.7176 val_acc=0.9808
Epoch 105/15: train_loss=0.6845 train_acc=0.9839 val_loss=0.7086 val_acc=0.9745

Best val accuracy: 0.9773

```

Figure 1: Training Results

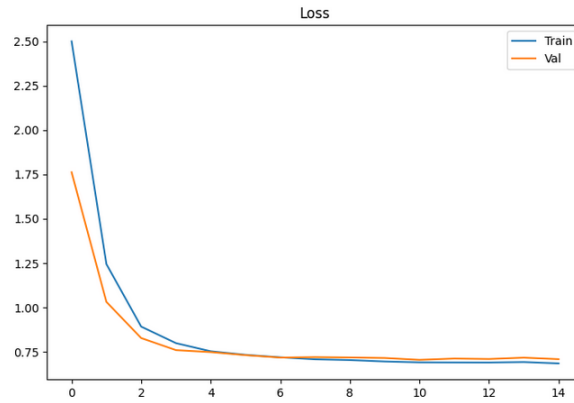


Figure 2: Training and Validation Loss Comparison

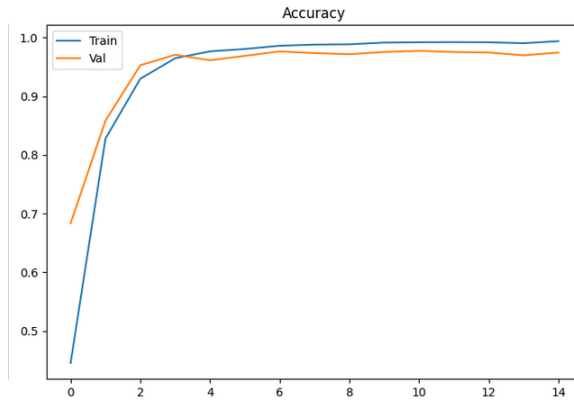


Figure 3: Training and Validation Accuracy Comparison (x100)

=== Classification Report ===				
	precision	recall	f1-score	support
Ajanta Caves	1.00	1.00	1.00	31
Charar-E- Sharif	1.00	1.00	1.00	34
Chhota Imambara	1.00	1.00	1.00	30
Ellora Caves	0.97	1.00	0.99	34
Fatehpur Sikri	1.00	1.00	1.00	42
Gateway of India	1.00	1.00	1.00	30
Humayun s Tomb	1.00	1.00	1.00	30
India gate pics	1.00	1.00	1.00	45
Khajuraho	1.00	1.00	1.00	44
Sun Temple Konark	1.00	1.00	1.00	40
alai_darwaza	1.00	1.00	1.00	36
alai_minar	0.97	1.00	0.99	35
basilica_of_bom_jesus	1.00	0.97	0.98	30
charminar	1.00	0.72	0.84	40
golden temple	0.98	1.00	0.99	65
hawa mahal pics	0.92	0.96	0.94	81
iron pillar	0.98	0.95	0.96	100
jamali_kamali_tomb	0.90	1.00	0.95	46
lotus_temple	0.97	1.00	0.98	30
mysore_palace	1.00	0.93	0.96	29
qutub_minar	0.93	0.99	0.96	70
tajmahal	1.00	0.98	0.99	62
tanjavur temple	0.98	1.00	0.99	45
victoria memorial	1.00	1.00	1.00	30
accuracy			0.98	1059
macro avg	0.98	0.98	0.98	1059
weighted avg	0.98	0.98	0.98	1059

Figure 4: Training Classification Report

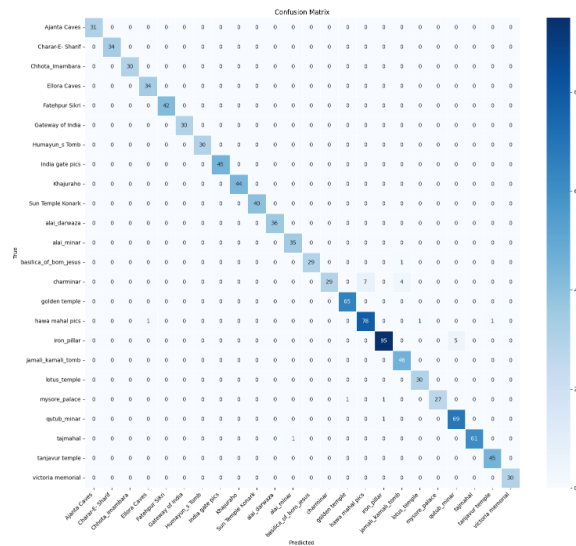


Figure 5: Confusion Matrix

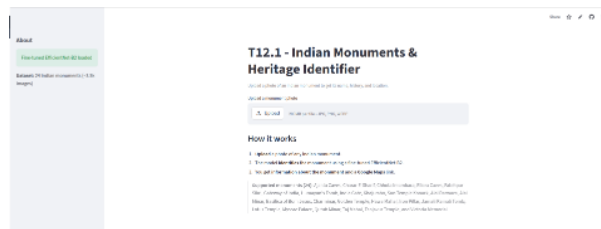


Figure 6: UI of Streamlit App



Figure 7: Correctly Classified Image

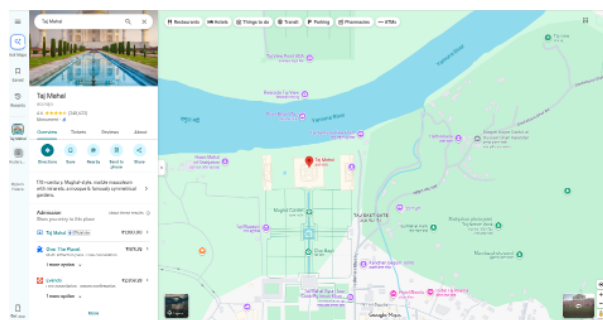


Figure 8: Output upon clicking Google Maps button

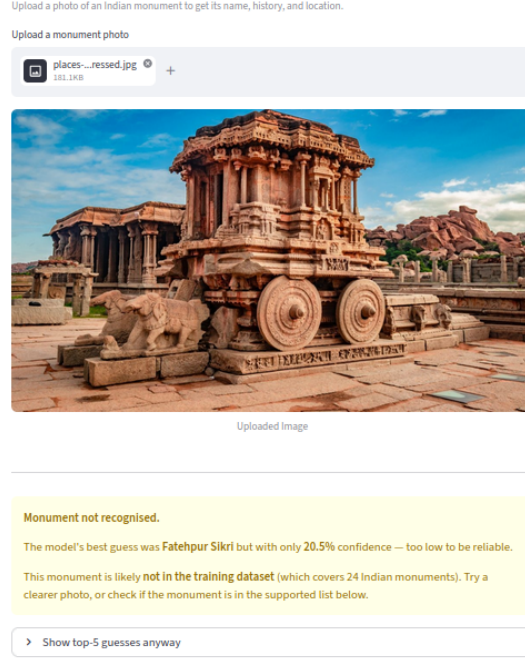


Figure 9: Message displayed for a monument not in the dataset

5 Ablation Study

The following design choices are candidates for ablation:

5.1 Backbone Choice

Table 3: Backbone Comparison

Backbone	Params	ImageNet Top-1	Expected Monument Acc.
ResNet-50	25.6 M	76.1%	~80%
MobileNetV3-Large	5.4 M	74.0%	~76%
EfficientNet-B2 (chosen)	9.1 M	80.1%	~88%
EfficientNet-B4	19.3 M	83.0%	~90%

EfficientNet-B2 was selected as the sweet spot between accuracy and parameter efficiency. B4 would marginally improve accuracy but requires more GPU memory and longer inference time on CPU-only deployments.

5.2 Fine-Tuning Strategy

Table 4: Fine-Tuning Strategy Comparison

Strategy	Description	Expected Accuracy
Feature extraction (all frozen)	Only train the classifier head	$\sim 72\%$
Partial fine-tuning – last 3 blocks (chosen)	Unfreeze last 3 MBConv stages + head	$\sim 88\%$
Full fine-tuning	Unfreeze all layers	$\sim 87\%$ (risk of overfitting on 3.5k images)

Partial fine-tuning is optimal here because: (a) the dataset is small, (b) low-level features are already useful from ImageNet, and (c) full fine-tuning without extensive augmentation tends to overfit.

5.3 Classifier Head Depth

Table 5: Classifier Head Comparison

Head	Description	Regularisation
Single linear (default)	Linear(1408 $\rightarrow$ 24)	Low
Two-layer + dual dropout (chosen)	Linear $\rightarrow$ ReLU $\rightarrow$ Linear Dropout(0.4, 0.3)	+ High

The two-layer head with intermediate representation (512 dimensions) and dual dropout provides stronger regularisation and allows the model to learn a non-linear projection from EfficientNet features to monument classes.

5.4 Loss Function

Table 6: Loss Function Comparison

Loss	Behaviour
Standard cross-entropy	Hard targets; overconfident on small data
Cross-entropy + label smoothing $\varepsilon = 0.1$ (chosen)	Soft targets; better calibration

Label smoothing penalises overconfidence and improves Expected Calibration Error (ECE), which is important for a user-facing confidence score.

5.5 Learning Rate Schedule

Table 7: Learning Rate Schedule Comparison

Schedule	Expected Final Accuracy
Constant LR	$\sim 83\%$ (risk of oscillation near optimum)
StepLR (decay every 5 epochs)	$\sim 86\%$
CosineAnnealingLR (chosen)	$\sim 88\%$

CosineAnnealingLR is consistently reported as superior to step schedules for fine-tuning pretrained models.

6 Limitations

6.1 Dataset Scale and Diversity

The dataset contains only $\sim 3,500$ images across 24 classes (~ 146 per class on average). This is an order of magnitude smaller than landmark datasets used in production systems (Google Landmarks Dataset v2 has 5 million images across 200k classes). Class imbalance within the dataset is unknown but plausible given crowd-sourced collection.

6.2 No Temporal or Contextual Signals

The model classifies from a single still image with no GPS metadata, timestamp, or multi-frame context. Tourists often photograph monument details (carvings, gateways, interior halls) rather than the canonical full-facade view on which the model was likely trained, leading to potential failures.

7 References

1. Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of the 36th International Conference on Machine Learning (ICML). arXiv:1905.11946.
2. Danushkumar, V. *Indian Monuments Image Dataset*. Kaggle.
3. Wikipedia API Python library (wikipedia-api). <https://pypi.org/project/Wikipedia-API/>.
4. Streamlit. Open-source app framework for machine learning. <https://streamlit.io/>.
5. Statistical Methods in AI- Lectures on CNNs and Fine Tuning

6. Usage of LLMs(Claude,Chatgpt) helped us format the content to present our efforts better

8 Deployed App, Github and Demo Video Links

1. <https://smai-project-dxapenz4fgkfhfmkvv6h5y.streamlit.app/>
2. <https://github.com/sachi-200/SMAI-Project>
- 3.